COS 214 PRACTICAL ASSIGNMENT 3

Danel Steyn, Deon Steenkamp and Logan van der Merwe

Contents

Table of Figures

Task 1

Task 1.1: Event concept

Our system models an event called **City Lights Festival.**

City Lights Festival is represented as a hierarchical event structure made up of larger event areas and smaller operational units. The root of the event is City Lights Festival itself. It contains Main Stage Zone and River Zone. Main Stage Zone also contains the nested Food Court group, which gives the Composite structure multiple levels of nesting.

The event contains the following concrete operational unit types:

1. Stage
2. Gate
3. Vendor
4. ShuttleStop
5. MedicalTeam
6. InformationDesk

These units have different responsibilities and different runtime behaviour. For example, Stage, can pause during unsafe conditions.
Gate, controls attendee admission.
Vendor,can suspend service.
ShuttleStop, can reroute transport.
MedicalTeam, remains operational during emergencies.
InformationDesk, provides information services.

The grouping structure used in the implemented event includes:

- City Lights Festival
- Main Stage Zone
- Food Court
- River Zone

EventGroup represents these grouping areas and can contain both individual event units and other EventGroup objects. This allows the complete event to be treated as a part-whole Composite hierarchy.

The Observer pattern is used separately from the ownership structure. EventControl can issue event notices, while selected EventGroup and event-unit objects register as observers. EventGroup can receive a notice from above and relay it to registered

observers below. This allows information such as weather warnings, capacity alerts, evacuation instructions and resume notices to travel through the event without EventControl needing to know the concrete type of every receiving object.

Task1.2 Identify the GoF participants of both Observer and Composite in your design

Composite Pattern

Component: EventComponent
EventComponent is the common abstraction for all objects that participate in the event structure. It declares the shared operations used by both individual units and groups.

Composite: EventGroup
EventGroup is the Composite participant. It owns a collection of std::unique_ptr<EventComponent> children and can therefore contain both concrete event units and other EventGroup objects. It recursively implements open(), close(), reportStatus() and getCapacity().

Leaf base class: EventUnit
EventUnit provides the common base for concrete event units.

Concrete Leaves: Stage, Gate, Vendor, ShuttleStop, MedicalTeam, InformationDesk
These classes represent individual operational parts of the event. They do not contain child EventComponents.

Client: main.cpp
The setup and integration code in main.cpp acts as the client that constructs the Composite tree and invokes operations on the event structure.

Observer Pattern

Subject: Subject
Subject is the GoF Subject abstraction. It stores non-owning Observer pointers and provides attach(), detach() and notify(). Observer registration is kept separate from Composite ownership.

Observer: Observer
Observer is the GoF Observer abstraction. It defines update(const Notice&), allowing concrete observers to react to notices polymorphically.

Concrete Subject: EventControl
EventControl acts as the event level concrete Subject. It is deliberately not part of the

Composite ownership tree. It stores a non-owning pointer to the root EventComponent and issues notices to registered observers using issueNotice().

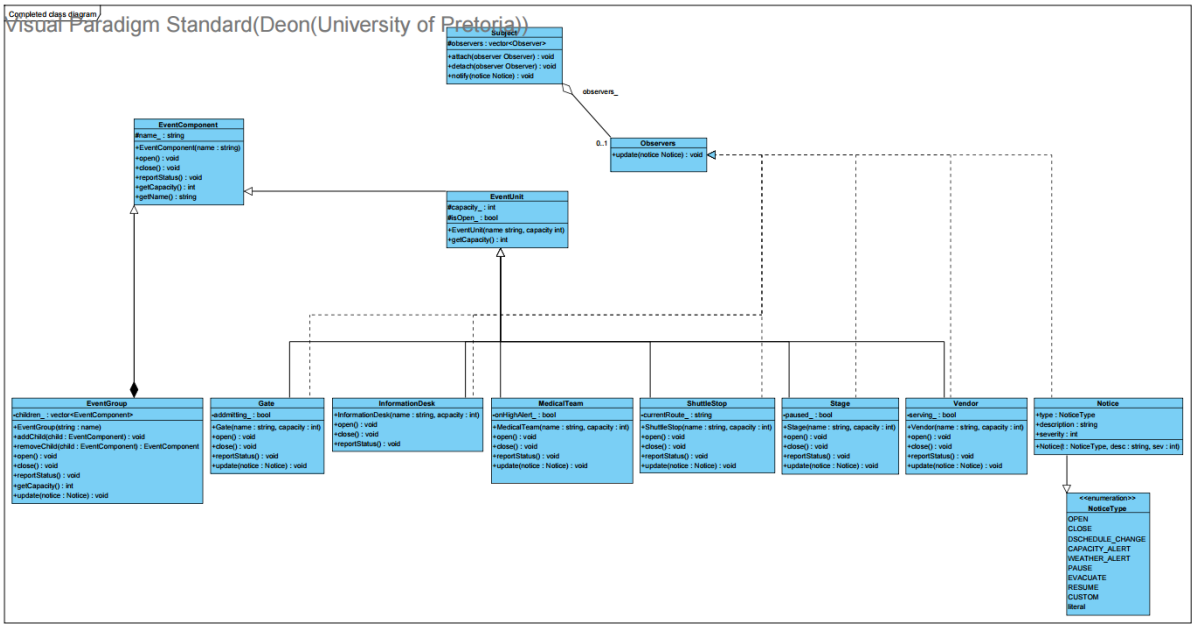
Concrete Observer and Subject: EventGroup
EventGroup participates in both patterns. It is a Composite in the structural hierarchy, but it also implements Observer and inherits Subject. When it receives update(const Notice&) from a Subject above it, it acts as an Observer. It then calls notify(const Notice&) to relay the notice to its own registered observers below, where it acts as a Subject.

Concrete Observers: Stage, Gate, Vendor, ShuttleStop, MedicalTeam
These classes react to notices through their own update(const Notice&) implementations.

InformationDesk is deliberately not an Observer. It participates only as a Composite Leaf because it has no meaningful notification reaction. This demonstrates that structural containment does not automatically imply Observer participation.

Notice: Notice
Notice is the data passed through the Observer collaboration. It contains the notice type, a description and a severity value. The implementation uses a push model, so the complete Notice is passed directly to update(const Notice&).

Task 1.3 UML class diagram



Task 1.4

a) Why your Composite is a genuine part-whole tree?

City Lights Festival contains event areas, those areas can contain smaller areas, and both kinds of groups may contain individual operational units. For example, Main Stage Zone contains Stage, Gate and the nested Food Court, while Food Court contains Vendor objects.This is therefore a genuine part-whole relationship rather than an ordinary association. Each child is structurally part of the EventGroup that owns it.EventGroup stores children as std::unique_ptr<EventComponent>, allowing the same container to own either another EventGroup or a concrete Leaf. Because both implement the EventComponent interface, operations such as open(), close(), reportStatus() and getCapacity() can be applied recursively without the client needing to know whether an object is a Composite or a Leaf.

(b) Why Observer is required rather than direct hard-coded calls?

A hard-coded notification structure would require EventControl to know every concrete unit type in the system and call each one by name (e.g. control.tellVendor(); control.tellMedic();). This breaks the moment a new unit type is added, and violates the practical's explicit rule against type-checking or class-aware dispatch chains. The Observer pattern decouples the broadcaster from the reactors entirely: EventControl (and any relaying EventGroup) only needs to hold a list of Observer* — it never needs to know whether that pointer refers to a VendorUnit, a MedicalUnit, or a type that doesn't exist yet. New unit types can be added without modifying Subject, EventControl, or any existing observer at all.

(d) Who owns or does not own observer references

Subject stores observers as non-owning raw pointers (std::vector<Observer*>). An observer's lifetime is governed entirely by whatever owns it in the Composite tree — never by the Subject it happens to be registered with. This means an observer must detach itself from any Subject it's registered with before its own destructor runs, to avoid the Subject holding a dangling pointer. Our policy: attach() is idempotent (registering an already-registered pointer is a silent no-op), and detach() on an unregistered pointer is also a safe no-op — this makes speculative detachment in a destructor always safe, regardless of whether the object was actually registered.

(e) Why EventGroup being both Subject and Observer is not a misuse of either pattern

EventGroup plays Observer with respect to its parent zone (it receives update() calls when a notice arrives from above) and Subject with respect to its own children (it issues notify() calls to relay that notice downward). These are two independent collaborations serving entirely different purposes: one governs *what this area is told*, the other governs *who this area tells*. The two roles never collapse into a single relationship — a class can implement Observer without ever being a Subject (e.g. a leaf MedicalUnit), and vice versa. EventGroup simply needs both roles because the scenario requires relay behaviour: a zone must both listen and rebroadcast.

Task 2

Task 2.3

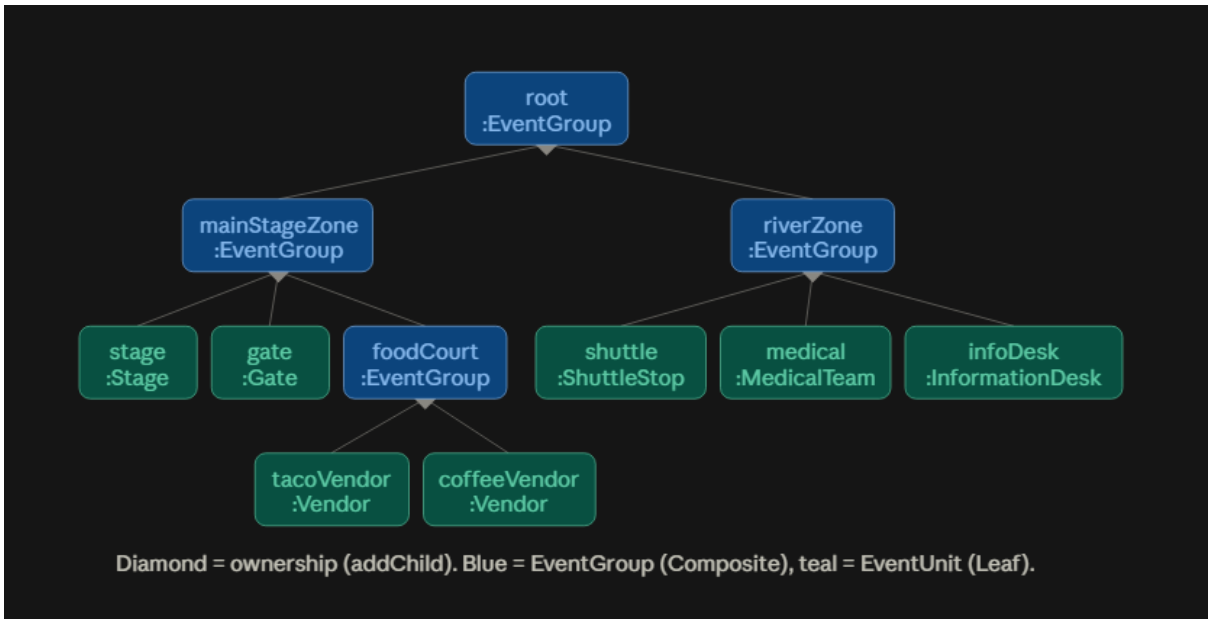


Figure 1Object diagram depicting EventFlow Composite ownership tree.

Task 2.4: Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once. If an object can be transferred between composites, define the ownership rules for that transfer clearly.

Single-owner invariant. Every `EventComponent` in the tree is owned by exactly one `std::unique_ptr` at all times — either sitting inside some `EventGroup::children_` vector, or, transiently during a transfer, held by the local variable that `removeChild()` returned it into. `unique_ptr` cannot be copied, only moved, so the compiler itself prevents two owners from ever existing simultaneously.

Destruction of the root. `EventGroup` declares no custom destructor, so the compiler-generated one destroys `children_` (a `vector<unique_ptr<EventComponent>>`) automatically. Destroying that vector destroys every `unique_ptr` in it, which destroys whatever each one owns — recursively, since a child that is itself an `EventGroup` repeats the same process one level down. The result: destroying `root` releases the entire owned subtree exactly once, with no explicit `delete` written anywhere in the codebase. This was verified with `valgrind --leak-check=full` on the full built tree: **46 allocations, 46 frees, 0 bytes in use at exit, 0 errors.**

Transfer rule. A unit moves between zones via `EventGroup::removeChild(EventComponent*)` — which finds the matching `unique_ptr`, moves it out of `children_`, erases the now-empty slot, and returns ownership to the caller — followed by `EventGroup::addChild(std::unique_ptr<EventComponent>)` on the destination zone, which moves it in. Between those two calls, the object is owned solely by the local variable holding the return value — never by zero zones, never by two. Demonstrated in `composite_test.cpp` by transferring `Coffee Vendor` from `Food Court` to `River Zone`; total event capacity was confirmed identical before and after (7210), and the corresponding Observer registration was explicitly `detach()`ed from `Food Court` and `attach()`ed to `River Zone` in the same operation, since the two collaborations are managed independently.

Task 3

Task 3.1: Subject abstraction and registration list

Subject maintains a non-owning std::vector<Observer*> registration list.

- **Duplicate registration policy:** attach() checks whether the pointer is already present before inserting; if so, it is a silent no-op. This prevents an observer being notified twice per broadcast due to accidental double-registration.
- **Detach-not-registered policy:** detach() on a pointer not currently in the list is a safe no-op — no exception, no error state. This allows destructors to call detach() speculatively on any Subject they might be registered with, without needing to track registration state separately.

Task 3.2: Observer abstraction and dangling-registration policy

Observer is a pure abstract interface (update(const Notice&), virtual destructor). Concrete observers (e.g. MedicalUnit, VendorUnit) implement update() with unit-specific reactions.

Lifetime policy: we do not rely on Subjects cleaning up after a deleted observer. Instead, each observer is responsible for calling detach() on every Subject it is registered with, before its own destructor completes. Because detach() is a safe no-op when the pointer isn't found, this is always safe to call even if the observer was never actually registered, or was already detached. This avoids dangling pointers in any Subject's registration list without requiring Subjects to track which observers still exist.

Task 3.3: Notice/order types

Implemented via the NoticeType enum, covering all three required categories:

- **Ordinary operational changes:** OPEN, CLOSE, SCHEDULE_CHANGE
- **Capacity-related change:** CAPACITY_ALERT
- **Safety-related changes:** WEATHER_ALERT, PAUSE, RESUME, EVACUATE
- **Extension point:** CUSTOM, for event-specific notices

Each Notice is a lightweight value type carrying type, a human-readable description, and an optional severity payload (e.g. capacity percentage, wind speed).

Task 3.4: Cascading notification through Composite

EventGroup (a Composite-level class) implements Observer, receiving update(const Notice&) from its parent zone or from EventControl. On receiving a notice, it immediately calls its own notify(notice) (inherited from Subject), relaying the notice to its own registered children. This is demonstrated across three runtime levels in SD2: EventControl → mainZone → foodCourt → tacoVendor, with mainZone also relaying directly to medicalTeam at the second level.

Task 3.5: Push vs pull (Why we chose to use push)

We chose push: Observer::update(const Notice&) carries the complete Notice payload (type, description, severity) directly, rather than requiring the observer to call back into the Subject to query state. This makes the exact information transferred at each cascade step explicit and visible directly on the sequence diagram arrows (see SD2) — a marker can trace exactly what data moved without inferring what a subsequent pull-query would have returned. It also avoids observers needing a back-reference to their Subject purely to fetch state, keeping the Observer interface minimal. The trade-off is that Subject::notify() must construct a reasonably complete Notice up front rather than letting each observer pull only the fields it needs — acceptable here since Notice is a small, fixed-shape value type with no expensive-to-compute fields.

Task 4: Event rules and dynamic behaviour

4.1. Define at least five event rules that cause concrete units to react differently to the same notice

1. **Stage safety rule:** When a `Stage` receives a `WEATHER_ALERT` or `EVACUATE` notice, the performance is paused. When a `RESUME` notice is received, the stage resumes its performance.
2. **Gate admission rule:** A `Gate` reacts to safety and capacity conditions by changing admission behaviour. During an evacuation it stops normal admission, while recovery through a `RESUME` notice restores admission.
3. **Vendor service rule:** A `Vendor` suspends service when an event condition requires operations to be restricted, such as a `CAPACITY_ALERT` or evacuation condition. A `RESUME` notice restores normal service.
4. **Shuttle transport rule:** A `ShuttleStop` does not simply shut down when conditions become unsafe. A `WEATHER_ALERT` causes the shuttle to use an alternative sheltered route, while an `EVACUATE` notice changes it to an emergency evacuation route. A `RESUME` notice restores the standard route.
5. **Medical response rule:** A `MedicalTeam` remains operational during event disruptions. On `EVACUATE` it raises its readiness state instead of closing, and on `RESUME` it stands down from high alert while remaining operational.

4.2. Implement at least one runtime reorganisation.

Runtime reorganisation is demonstrated by transferring the **Coffee Vendor** from the **Food Court** to the **River Zone** while the event is running.

The operation is implemented by `EventRules::transferUnit(...)`. The source `EventGroup` first releases its `std::unique_ptr<EventComponent>` using `removeChild()`. Ownership is then moved into the destination group's `children_` collection using `addChild()`.

The Observer relationship is updated together with the structural move. The transferred unit is detached from the old group's notification list and attached to the destination group's notification list. Therefore two separate relationships are changed correctly:

- **Composite relationship:** the River Zone becomes the new owner of Coffee Vendor.
- **Observer relationship:** Coffee Vendor stops receiving notifications through Food Court and begins receiving notifications through River Zone.

The transfer preserves the single-owner rule because the `unique_ptr` is moved rather than copied. The event's total capacity therefore remains unchanged: the Coffee Vendor still exists exactly once; only its location in the Composite tree changes.

4.3. Implement at least one condition-based decision that will later appear inside an alt or opt combined fragment,

A capacity threshold provides the main condition-based decision: **capacity >= threshold**

The predicate is implemented through EventRules::isOverCapacityThreshold(...) . It obtains the aggregate capacity of an EventGroup and compares it with a configured threshold. In the integrated demonstration, Main Stage Zone has an aggregate capacity of 7080 and the configured threshold is 7000. Therefore: **7080 >= 7000** evaluates to true, and EventControl broadcasts a CAPACITY ALERT.

The condition is deliberately implemented as a separate predicate rather than being hard-coded into a particular event unit. This also allows the same decision to be represented directly in a UML `alt` or `opt` combined fragment using a guard such as: [capacity >= threashold]

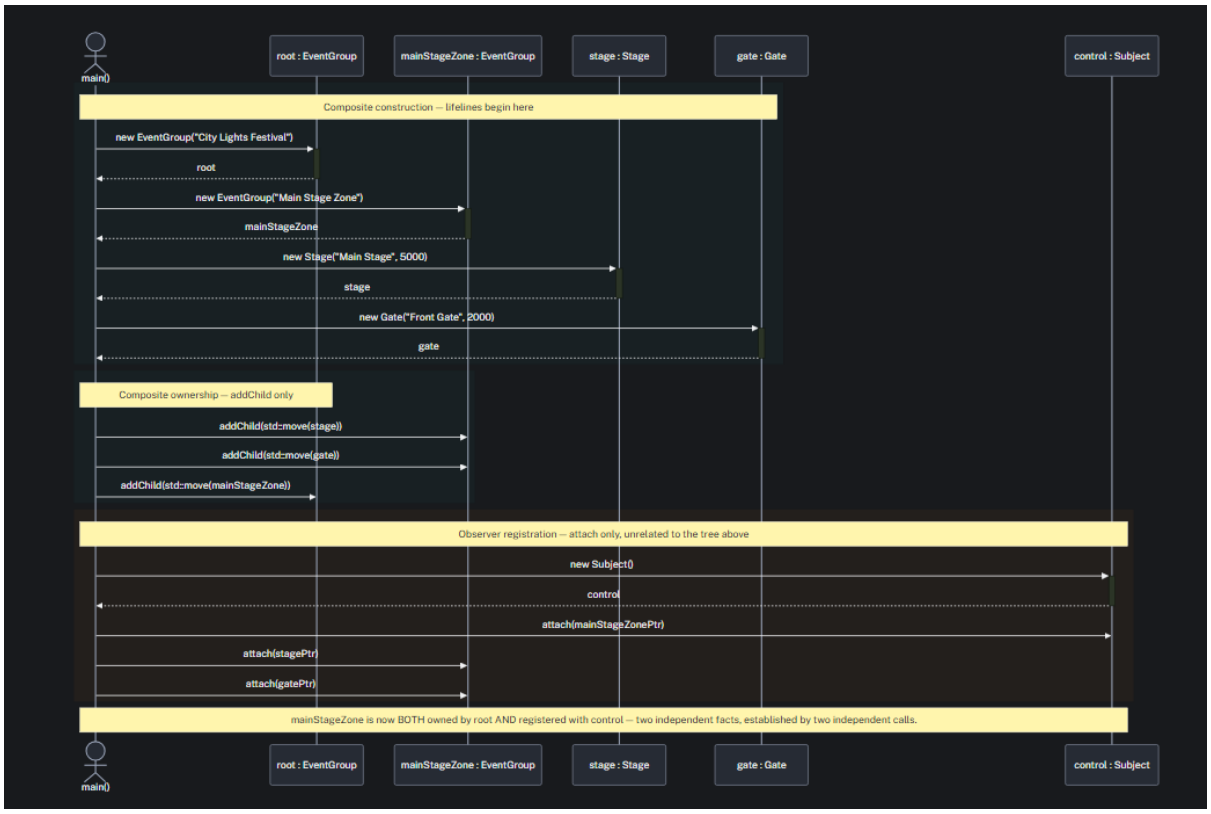
4.4. 3 Original features:

1. **Weather severity auto-escalation**. EventRules::deriveWeatherNotice() uses the measured wind speed and a configured evacuation threshold to determine which type of notice should be issued.
   If the wind speed is below the evacuation threshold, the function creates a WEATHER_ALERT. If the wind speed reaches or exceeds the threshold, it creates an EVACUATE notice.
   This is demonstrated in main.cpp using a wind speed of 75 km/h, which produces a WEATHER_ALERT, and later a wind speed of 105 km/h, which produces an EVACUATE notice.
   This feature belongs in EventRules because it is a decision rule based on event conditions. EventControl remains responsible only for issuing notices and does not need to contain weather-specific decision logic.
2. **Gate VIP fast-track mode.** The Gate contains a specialised VIP-only admission mode that can be activated when it receives a CAPACITY_ALERT. This provides the Gate with an alternative operational state rather than limiting it to only normal admission or complete closure.
   The behaviour belongs inside Gate because admission control is specific to a gate. EventControl only broadcasts the CAPACITY_ALERT and does not need to know how a Gate responds to it. This keeps the concrete reaction encapsulated in the class responsible for that behaviour.
3. **Shuttle route-history log.** ShuttleStop maintains a history of its route changes. When event conditions cause the shuttle to change routes, the new route is recorded so that the sequence of routing decisions can be inspected later.
   For example, the ShuttleStop can change from its Standard Route to an Indoor Shelter Route during a WEATHER_ALERT, to an Emergency Evacuation Route during an EVACUATE notice, and back to the Standard Route after a RESUME notice.
   This feature belongs inside ShuttleStop because route information and route history are specific to the transport unit. EventControl and EventGroup do not need to know how the shuttle stores or manages its route history.

These features do not create a god object because the responsibilities are distributed according to the classes that logically own the behaviour. EventRules handles event-condition decisions, Gate manages its own admission behaviour, and ShuttleStop manages its own transport behaviour and route history. EventControl remains a coordinator and broadcaster rather than becoming responsible for every event-specific rule.

Task 5

Task 5.2)



Task 5.2

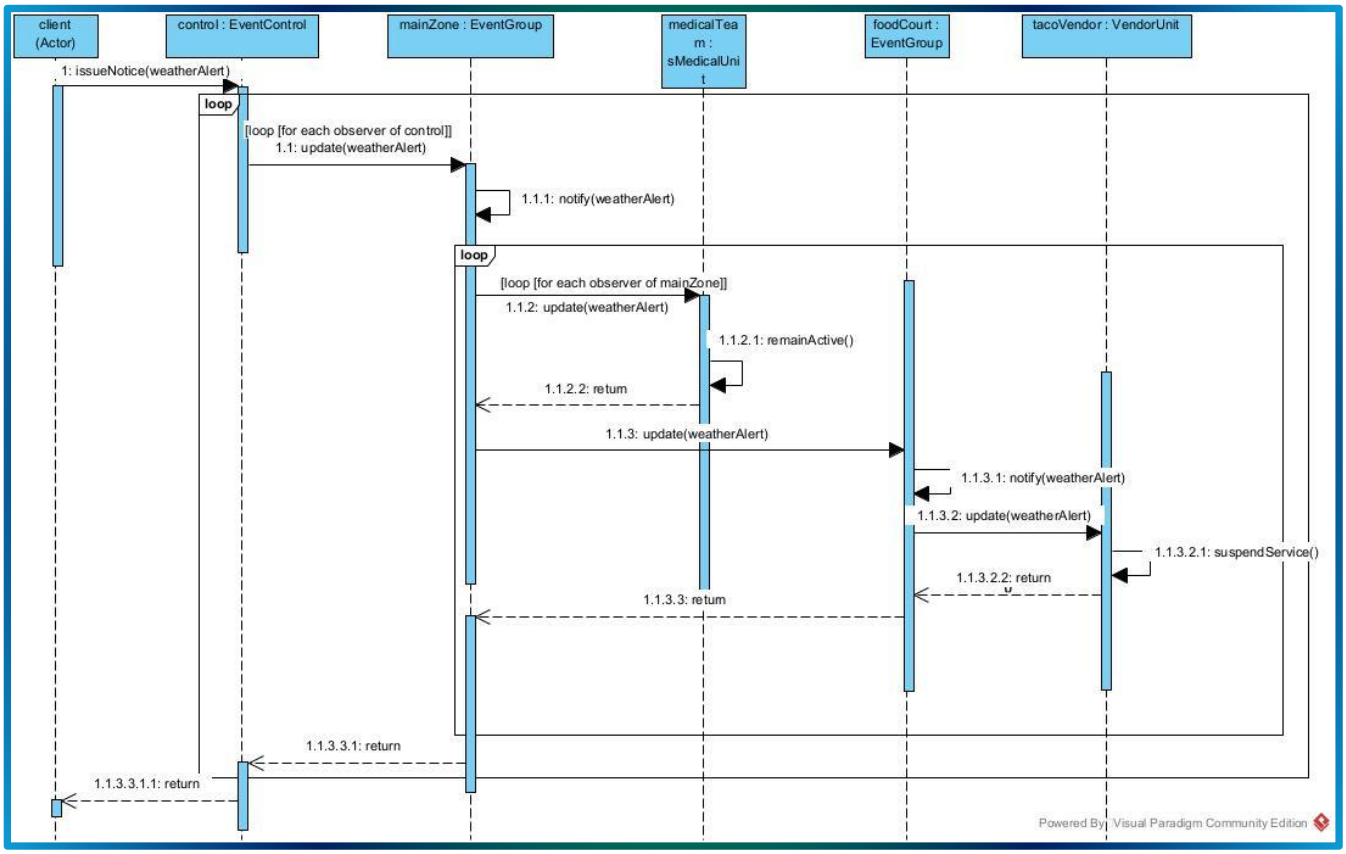
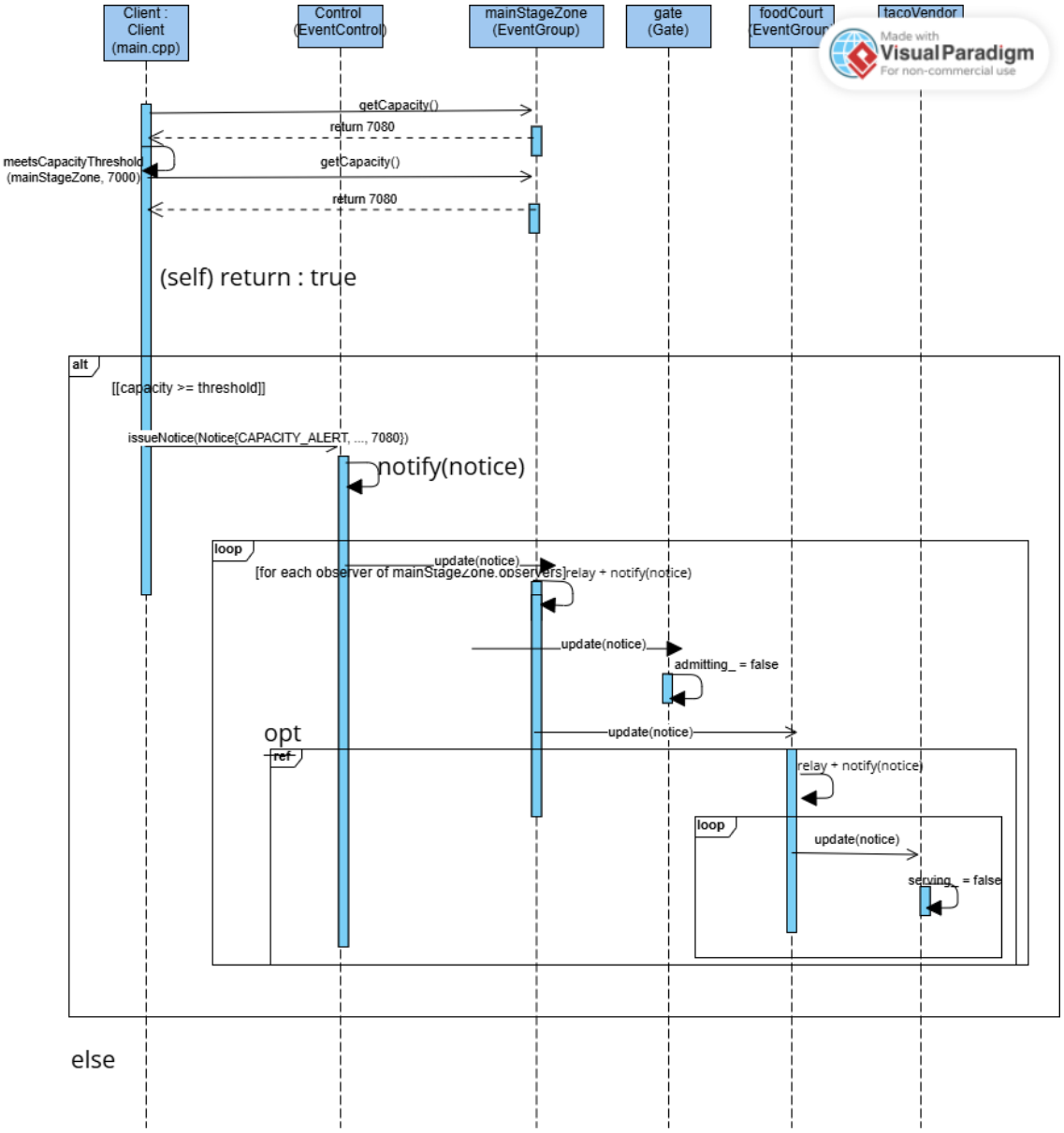
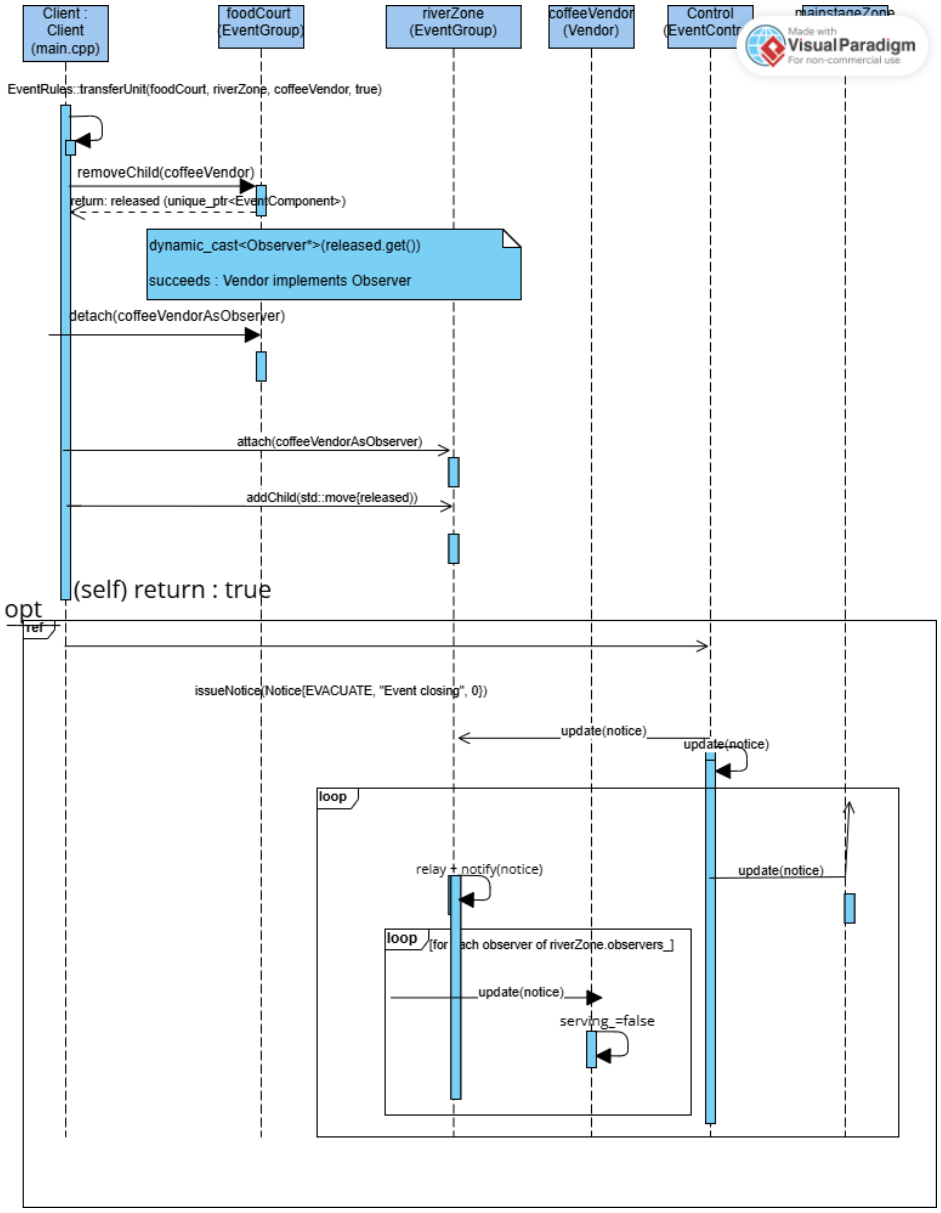


Figure 2: SD2: Cascading event notification. A WEATHER_ALERT notice originates at control : EventControl and cascades through mainZone : EventGroup to two of its observers, medicalTeam : MedicalUnit and foodCourt : EventGroup. foodCourt, itself both Observer and

Task 5.3


Client : Client (main.cpp)
Control (EventControl)
mainStageZone (EventGroup)
gate (Gate)
foodCourt (EventGroup)
tacoVendor
Made with VisualParadigm For non-commercial use
getCapacity()
return 7080
meetsCapacityThreshold (mainStageZone, 7000)
getCapacity()
return 7080
(self) return : true
alt
[[capacity >= threshold]]
issueNotice(Notice{CAPACITY_ALERT, ..., 7080})
notify(notice)
loop
update(notice)
[for each observer of mainStageZone.observers]relay + notify(notice)
update(notice)
admitting_ = false
opt
ref
update(notice)
relay + notify(notice)
loop
update(notice)
serving_ = false
else

Task 5.4


Client : Client (main.cpp)
foodCourt (EventGroup)
riverZone (EventGroup)
coffeeVendor (Vendor)
Control (EventContr
Made with VisualParadigm For non-commercial use
EventRules::transferUnit(foodCourt, riverZone, coffeeVendor, true)
removeChild(coffeeVendor)
return: released (unique_ptr<EventComponent>)
dynamic_cast<Observer*>(released.get())
succeeds : Vendor implements Observer
detach(coffeeVendorAsObserver)
attach(coffeeVendorAsObserver)
addChild(std::move(released))
(self) return : true
opt
ref
issueNotice(Notice{EVACUATE, "Event closing", 0})
update(notice)
update(notice)
loop
relay + notify(notice)
update(notice)
loop
update(notice)
serving_=false

Task 6: Doxygen Documentation

6.1

Class-level Doxygen added to all four classes, each stating its GoF role (Observer abstraction, Subject abstraction, concrete Subject/coordinator).

6.2

All public operations documented with @param/@return where applicable; attach/detach explicitly document that they take non-owning pointers.

6.3

Non-obvious design decisions documented in comments: (1) why Subject stores non-owning pointers, (2) why attach/detach are idempotent/safe-no-op rather than erroring, (3) why notify() iterates over a snapshot rather than the live list.

6.4

Confirmed doxygen Doxyfile generates cleanly with zero warnings on Notice.h, Observer.h, Subject.h/.cpp, EventControl.h/.cpp.

Task 7. GitHub Workflow Reflection

Our team used a shared GitHub repository and divided the implementation according to the main areas of the practical. Deon focused primarily on the Composite structure and concrete event units, Logan focused on the Observer notification system and EventControl, and Danel focused on event rules, dynamic behaviour and final integration.

Development was performed on separate branches rather than by independently creating final files and combining them only at submission time. The Composite work was developed on the Composite/Structure branch, the Observer and notification work was developed on the Observer/-Notification branch, and the behaviour and integration work was developed on the feature/behaviour-integration branch.

The branches were integrated progressively. For example, the Composite/Structure branch was merged into feature/behaviour-integration so that the EventRules implementation could operate on the actual EventGroup and EventComponent classes. This merge produced conflicts in Notice.h, Observer.h, Subject.h and Subject.cpp because both branches contained versions of these shared files. The conflicts were inspected and resolved before the merge was completed rather than simply replacing one member's work.

Meaningful repository activity includes the implementation of runtime transfer and event-condition rules on the behaviour branch, the merge of the Composite structure into the behaviour integration branch, and the later integration of the Observer/EventControl implementation. The final integrated program was compiled and executed before the behaviour branch was merged into main.

Using branches was useful because each member could work on their own responsibility without continuously overwriting another member's files. The merge process also exposed integration problems while development was still taking place. Resolving these conflicts and testing the combined implementation helped ensure that the final Composite, Observer and dynamic behaviour code worked together rather than existing as three isolated solutions.